📱 Mobile APP Mapping and Navigation

1. Course Content

- Master using the [ROSRobot] mobile APP on mobile terminals (Android / iOS) to wirelessly remote control the A1 Pro robot for real-time environment mapping, map saving and loading.
- Master receiving real-time video stream, observing occupancy raster map and laser point cloud, calibrating initial pose via touch screen, and sending navigation goal points for autonomous cruising on the mobile APP.

---

2. Preparation

1. **Install APP**: Search and install `ROSRobot` in the mobile app store (or scan the QR code in the manual to download and install).
2. **LAN Connection**: Ensure the phone and robot are connected to the same WiFi LAN (or connect to the robot's hotspot WiFi), and record the robot's current IP address (can be viewed by executing `ifconfig` in terminal).

---

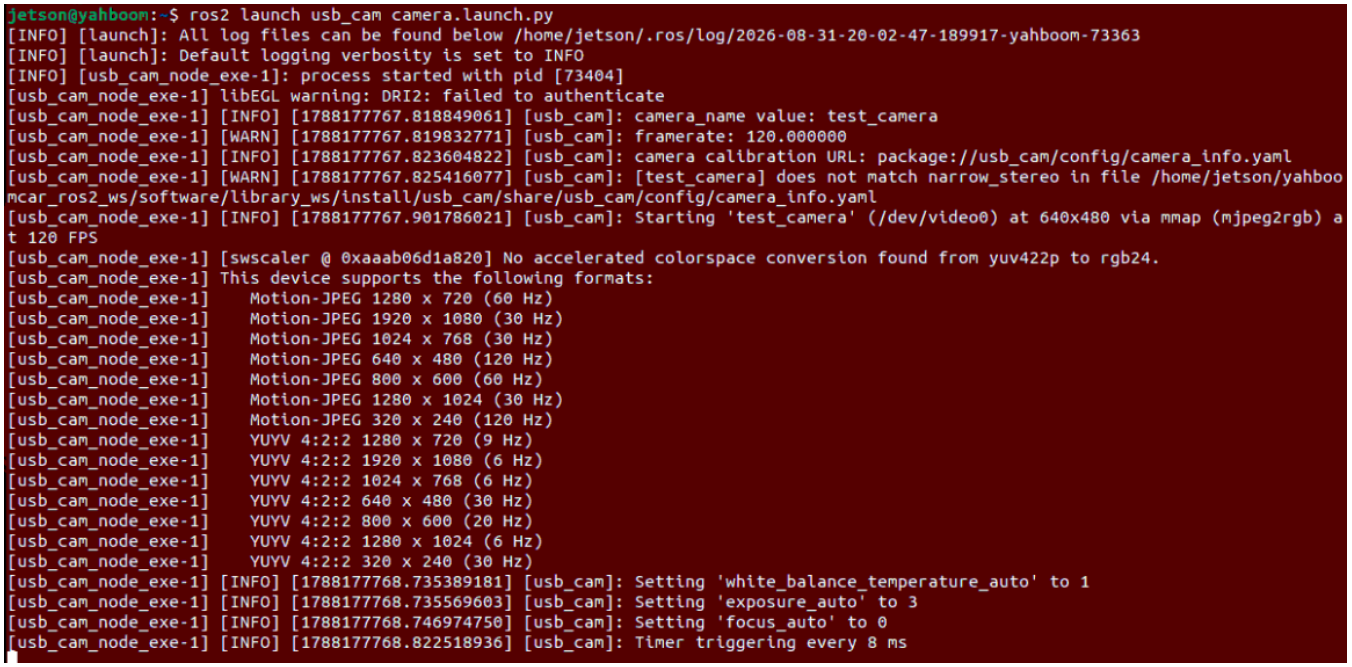
3. Mobile APP Mapping Operation

3.1 Start Camera Video Streaming Node

Based on the camera type used, start the corresponding camera node in **Terminal 1**:

```
# If using depth camera:
ros2 launch yahboomcar_depth camera_app.launch.py

# If using ordinary USB monocular / robotic arm camera:
ros2 launch usb_cam camera.launch.py
```



3.2 Start APP Mapping Service (Choose One)

Run in **Terminal 2** the APP mapping bridge node (this Launch automatically pulls up chassis driver, lidar, SLAM mapping node, Rosbridge WebSocket bridge, and APP map save service):

```
# Method A: Gmapping algorithm mapping (recommended)
ros2 launch yahboomcar_nav map_gmapping_app_launch.xml

# Method B: Cartographer algorithm mapping
ros2 launch yahboomcar_nav map_cartographer_app_launch.xml
```

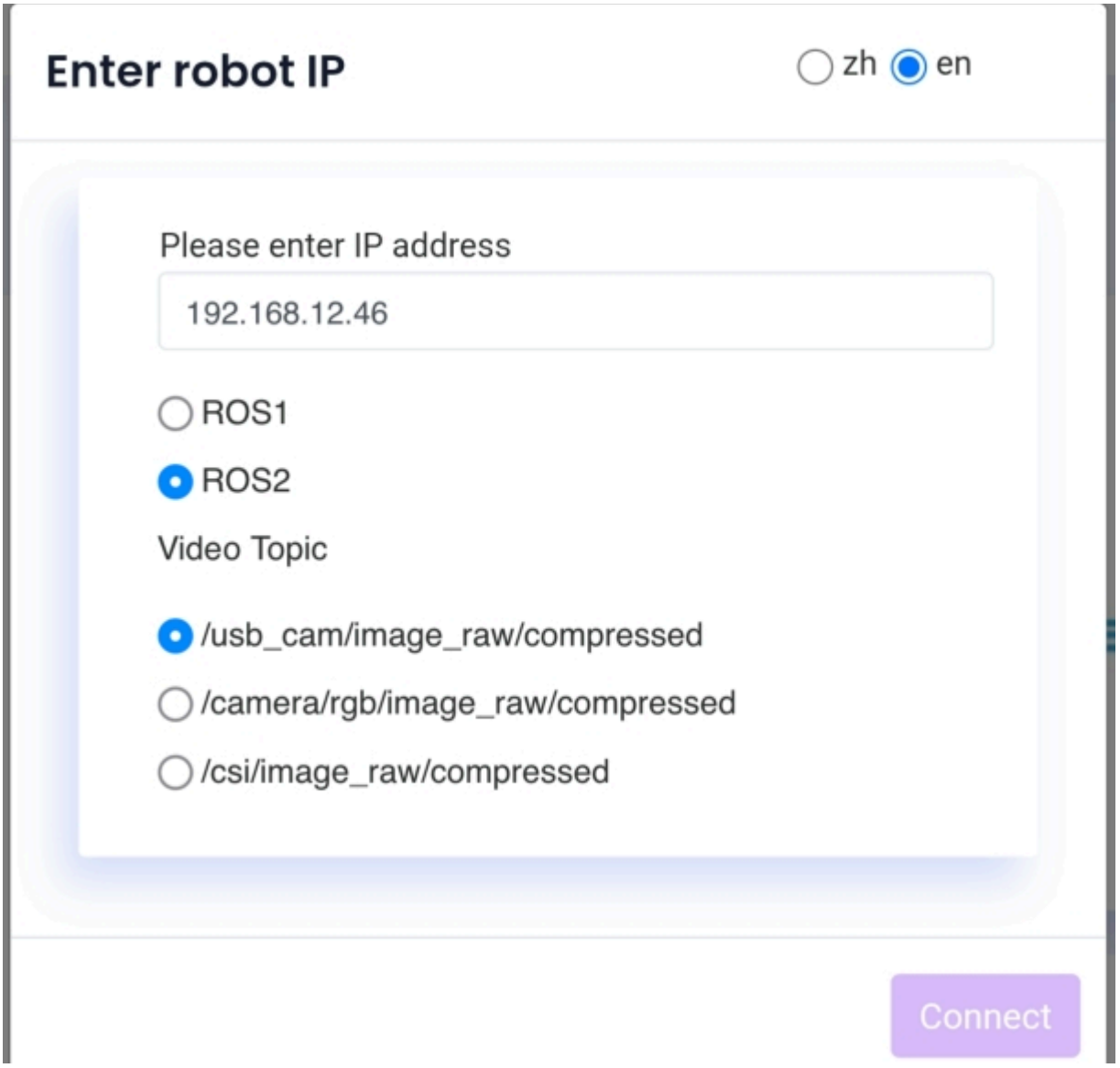
```
jetson@yahboom:~$ ros2 launch yahboomcar_nav map_gmapping_app_launch.xml
[INFO] [launch]: All log files can be found below /home/jetson/.ros/log/2026-08-31-20-03-17-648491-yahboom-74661
[INFO] [launch]: Default logging verbosity is set to INFO

-------- robot_type = A1P, rplidar_type = tmini, camera_type = usb --------

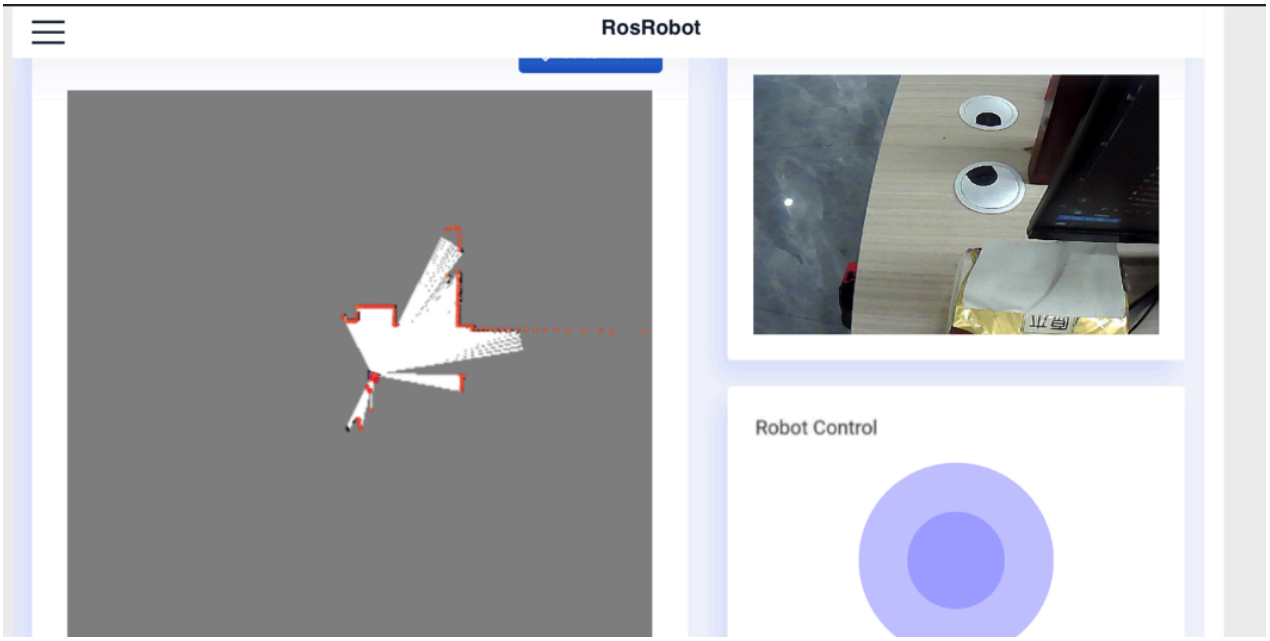
--------------------robot_type = A1--------------------
[INFO] [ydlidar_ros2_driver_node-10]: process started with pid [74789]
[INFO] [robot_pose_publisher-14]: process started with pid [74832]
[ydlidar_ros2_driver_node-10] [YDLIDAR] SDK initializing
[ydlidar_ros2_driver_node-10] [YDLIDAR] SDK has been initialized
[ydlidar_ros2_driver_node-10] [YDLIDAR] SDK Version: 1.2.3
[INFO] [rosbridge_websocket-1]: process started with pid [74760]
[INFO] [rosapi_node-2]: process started with pid [74762]
[INFO] [laserscan_to_point_publish-3]: process started with pid [74764]
[INFO] [joint_state_publisher-4]: process started with pid [74766]
[INFO] [robot_state_publisher-5]: process started with pid [74770]
[INFO] [Ackman_driver_A1P-6]: process started with pid [74772]
[INFO] [base_node_A1-7]: process started with pid [74775]
[INFO] [imu_attitude_converter-8]: process started with pid [74777]
[INFO] [ekf_node-9]: process started with pid [74786]
[INFO] [static_transform_publisher-11]: process started with pid [74798]
[INFO] [static_transform_publisher-12]: process started with pid [74802]
[INFO] [slam_gmapping-13]: process started with pid [74805]
[INFO] [server-15]: process started with pid [74834]
[robot_state_publisher-5] [INFO] [1788177798.833420561] [robot_state_publisher]: got segment Gripping
[robot_state_publisher-5] [INFO] [1788177798.833659224] [robot_state_publisher]: got segment arm1
[robot_state_publisher-5] [INFO] [1788177798.833683929] [robot_state_publisher]: got segment arm2
[robot_state_publisher-5] [INFO] [1788177798.833696185] [robot_state_publisher]: got segment arm3
[robot_state_publisher-5] [INFO] [1788177798.833705497] [robot_state_publisher]: got segment arm4
[robot_state_publisher-5] [INFO] [1788177798.833714073] [robot_state_publisher]: got segment arm5
[robot_state_publisher-5] [INFO] [1788177798.833722650] [robot_state_publisher]: got segment arm_base_Link
[robot_state_publisher-5] [INFO] [1788177798.833731898] [robot_state_publisher]: got segment base_footprint
[robot_state_publisher-5] [INFO] [1788177798.833740762] [robot_state_publisher]: got segment base_link
[robot_state_publisher-5] [INFO] [1788177798.833750426] [robot_state_publisher]: got segment camera_link
[robot_state_publisher-5] [INFO] [1788177798.833759675] [robot_state_publisher]: got segment imu_link
[robot_state_publisher-5] [INFO] [1788177798.833768699] [robot_state_publisher]: got segment laser_Link
[robot_state_publisher-5] [INFO] [1788177798.833777627] [robot_state_publisher]: got segment lbw_Link
[robot_state_publisher-5] [INFO] [1788177798.833785947] [robot_state_publisher]: got segment lfr_Link
[robot_state_publisher-5] [INFO] [1788177798.833794652] [robot_state_publisher]: got segment lfw_Link
[robot_state_publisher-5] [INFO] [1788177798.833802972] [robot_state_publisher]: got segment llink1
[robot_state_publisher-5] [INFO] [1788177798.833811548] [robot_state_publisher]: got segment llink2
[robot_state_publisher-5] [INFO] [1788177798.833820028] [robot_state_publisher]: got segment llink3
[robot_state_publisher-5] [INFO] [1788177798.833828285] [robot_state_publisher]: got segment rbw_Link
[robot_state_publisher-5] [INFO] [1788177798.833836957] [robot_state_publisher]: got segment rfr_Link
```

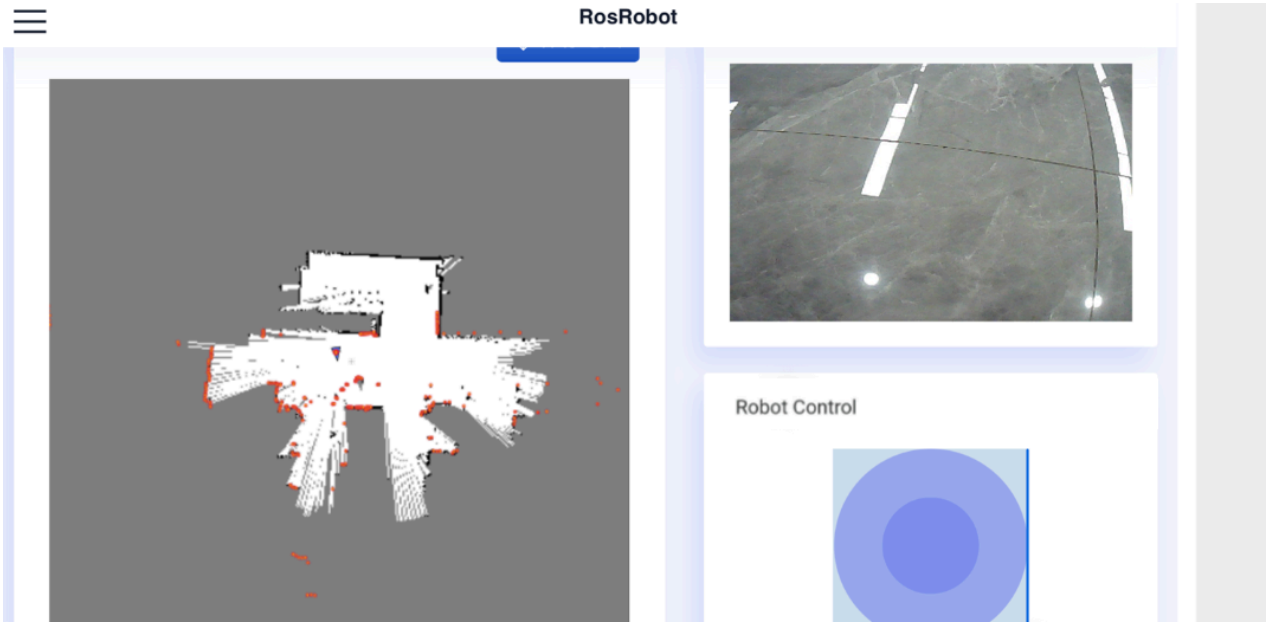
3.3 APP Connection and Mapping Operation

1. **Connect to Robot**:
   Open the `ROSRobot` APP on the phone, select mapping mode. Enter the robot's LAN IP address, keep the port as default `9090`, VideoTopic select `/usb_cam/image_raw/compressed` (if using Nuwa camera, select the corresponding compressed topic), and click **【Connect】**.

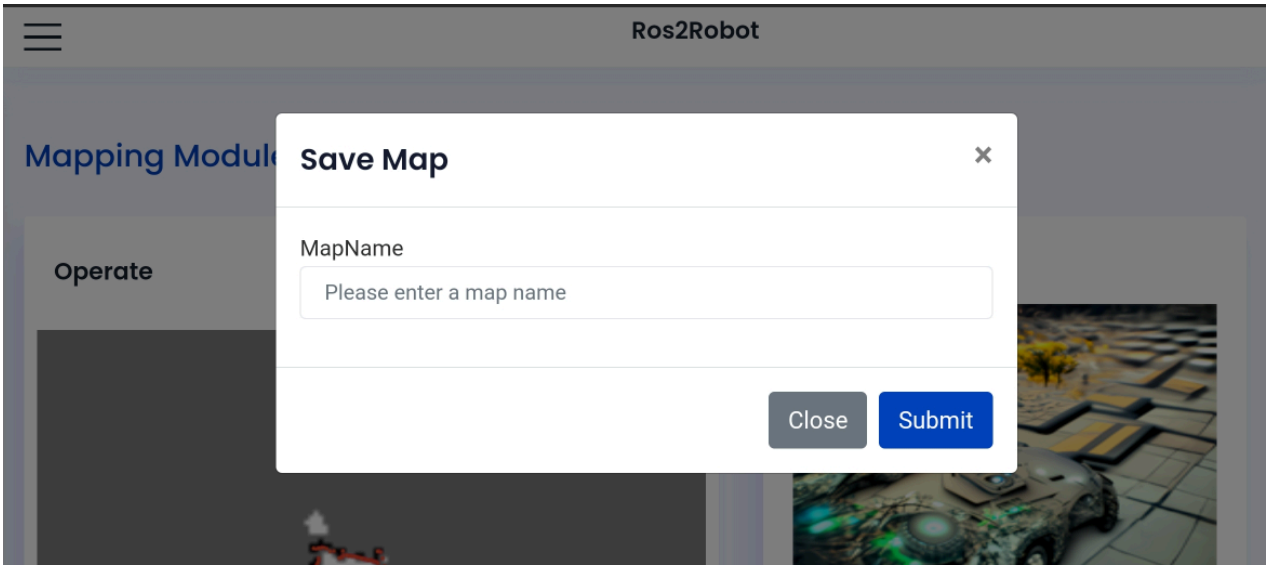


2. **Remote Control Mapping**:
   After successful connection, the APP interface will display the real-time camera video feed and the raster map generated by the lidar. Use the virtual joystick in the lower-left corner to remote control the robot to move slowly and smoothly, gradually scanning and exploring the entire indoor environment.



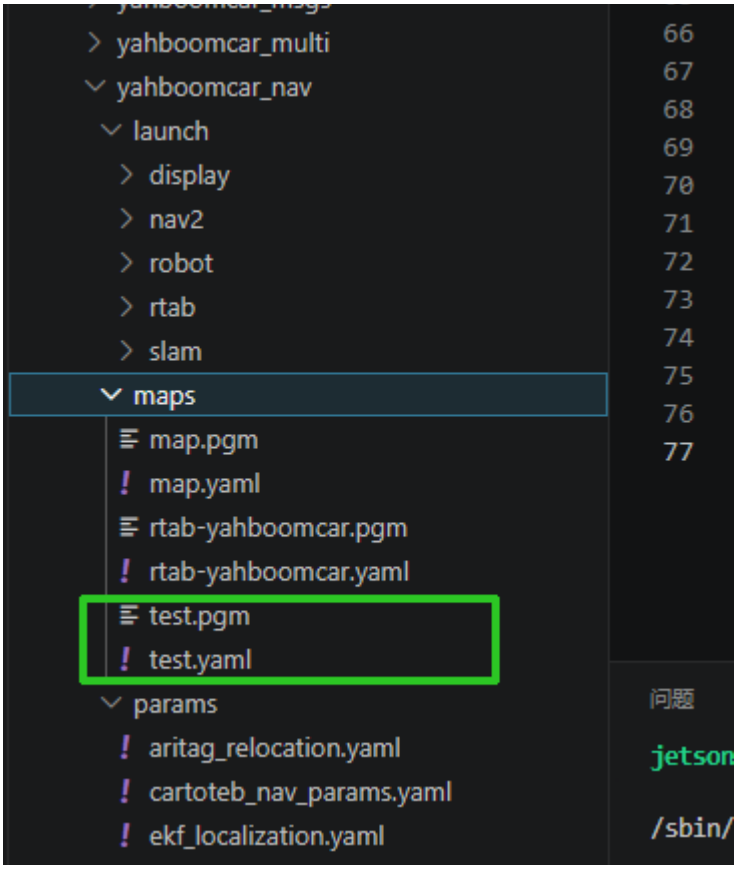


3. **Save Map**:
   When the environment outline is completely closed, click the 【**Save Map**】 button in the upper-right corner of the APP interface, enter the map name (for example `test`) in the popup, and click confirm.



4. **View Saved Maps**:
   The APP map save service will automatically call `map_saver_cli` in the background to save the map file to the robot system path:

```
ls ~/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_nav/maps/
```

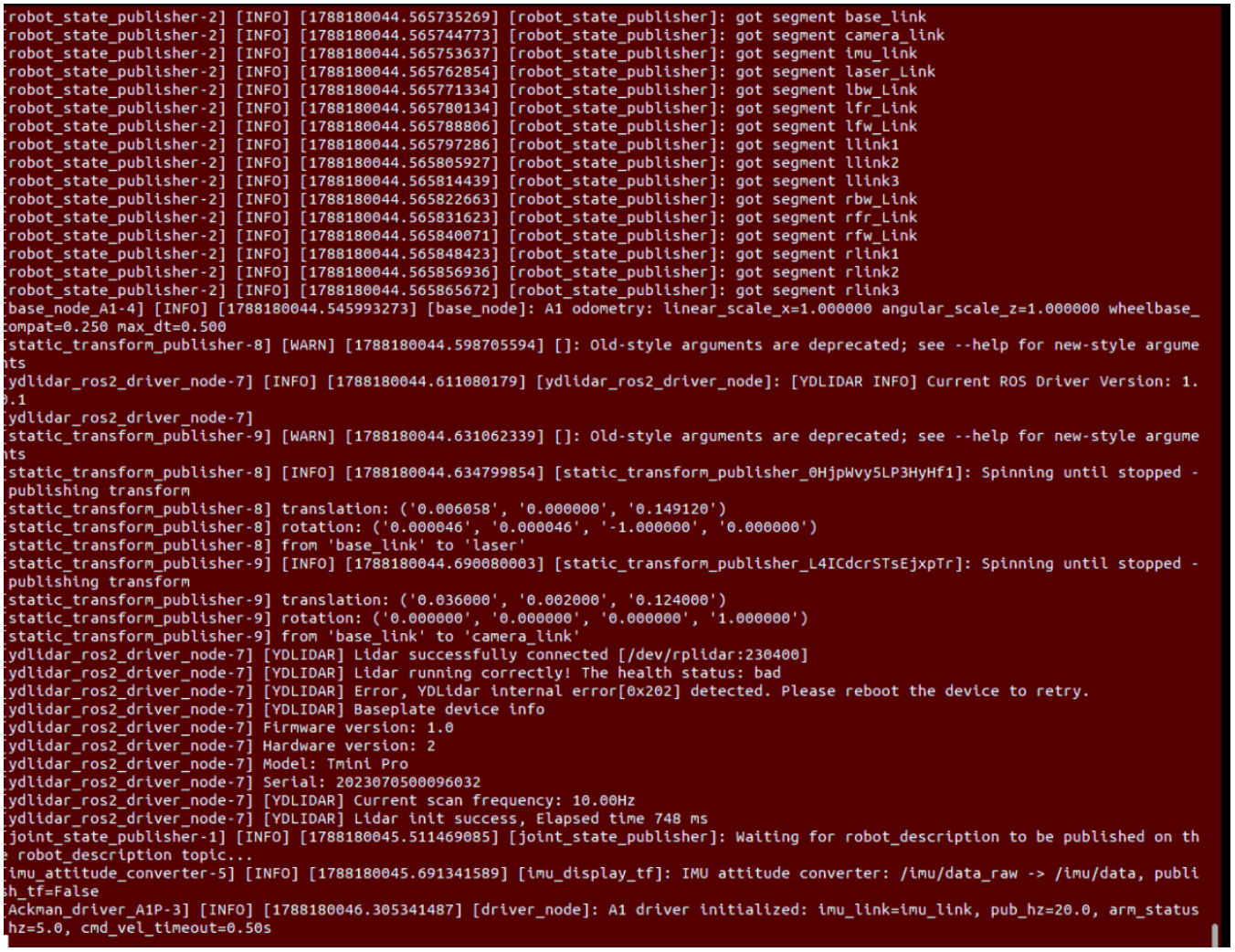


4. Mobile APP Autonomous Navigation Operation

4.1 Start Chassis and Lidar

Run in **Terminal 1** (start A1 Pro chassis driver and lidar):

```
ros2 launch yahboomcar_nav laser_bringup_launch.py
```

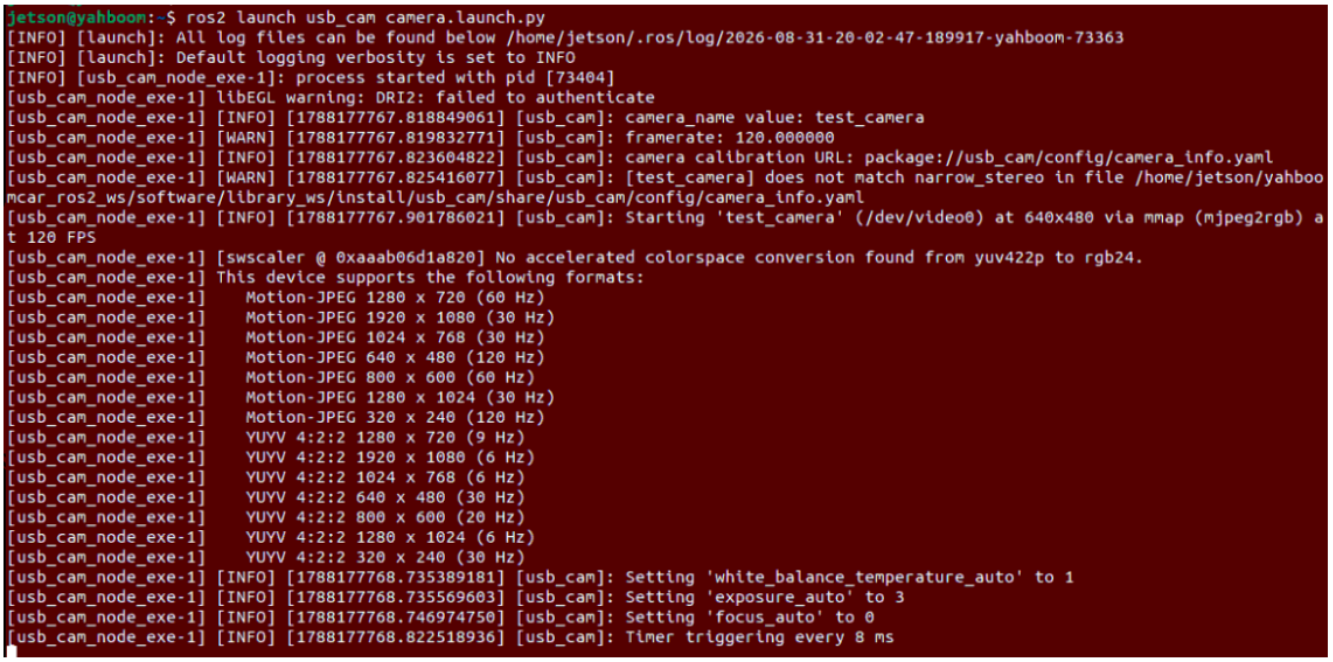


4.2 Start Camera Video Streaming Node

Run in **Terminal 2** the camera node:

```
# Nuwa camera:
ros2 launch yahboomcar_depth camera_app.launch.py

# USB camera:
ros2 launch usb_cam camera.launch.py
```



4.3 Start APP Navigation Service

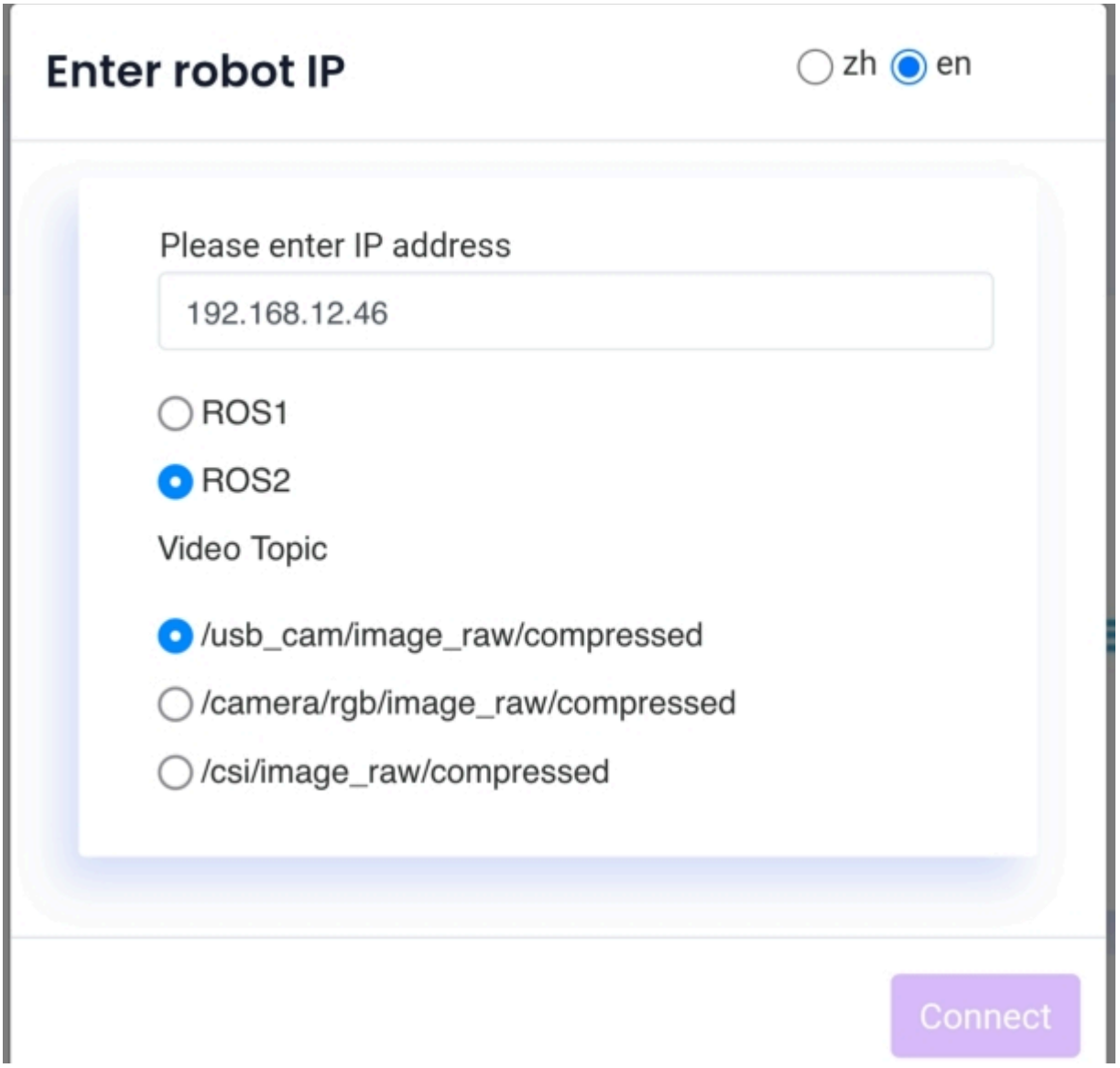
Run in **Terminal 3** the navigation node:

```
#map:=The path needs to be modified according to the actual mainboard
ros2 launch yahboomcar_nav navigation_teb_app_launch.xml
map:=/home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_nav/maps/test.yaml
```

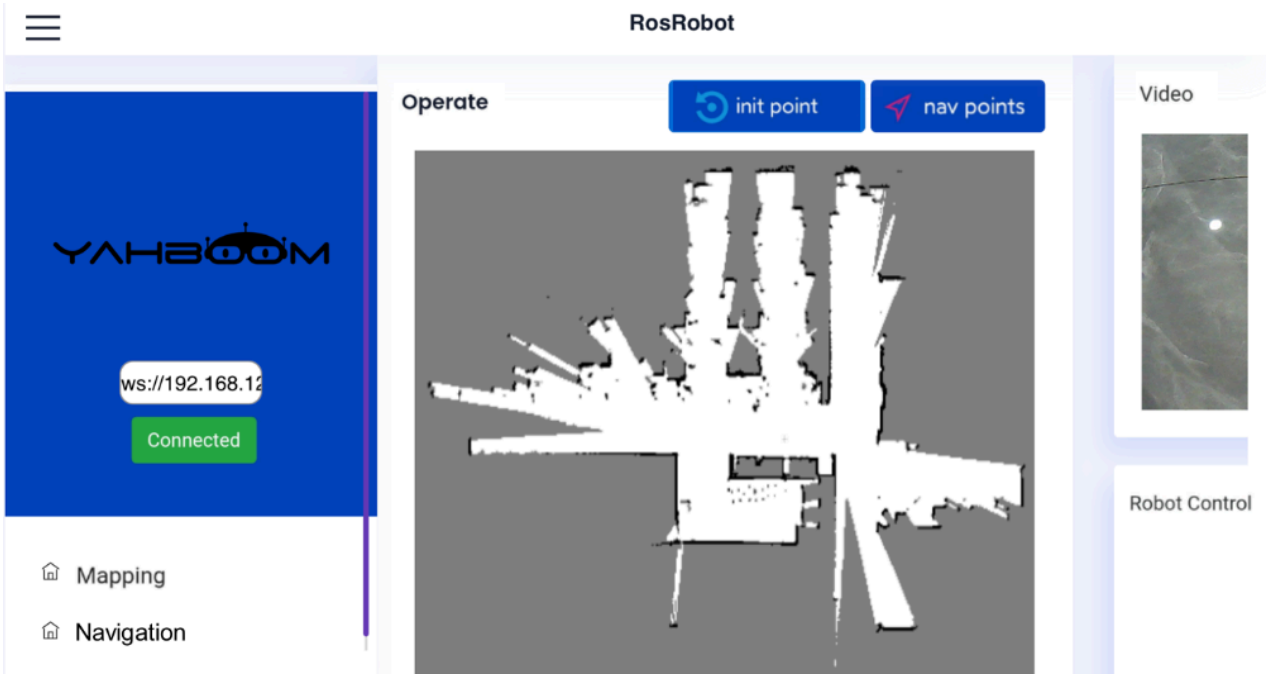
```
[component_container_isolated-4] [INFO] [1788180208.370757604] [lifecycle_manager_localization]: Configuring amcl
[component_container_isolated-4] [INFO] [1788180208.371965980] [amcl]: Configuring
[component_container_isolated-4] [INFO] [1788180208.372169280] [amcl]: initTransforms
[INFO] [launch_ros.actions.load_composable_nodes]: Loaded node '/planner_server' in container '/nav2_container'
[component_container_isolated-4] [INFO] [1788180208.390464715] [nav2_container]: Load Library: /opt/ros/humble/lib/libbehavior_serve
r_core.so
[component_container_isolated-4] [INFO] [1788180208.455849820] [nav2_container]: Found class: rclcpp_components::NodeFactoryTemplate
<behavior_server::BehaviorServer>
[component_container_isolated-4] [INFO] [1788180208.456018528] [nav2_container]: Instantiate class: rclcpp_components::NodeFactoryTe
mplate<behavior_server::BehaviorServer>
[component_container_isolated-4] [INFO] [1788180208.479895706] [amcl]: initPubSub
[component_container_isolated-4] [INFO] [1788180208.489264756] [behavior_server]:
[component_container_isolated-4]        behavior_server lifecycle node launched.
[component_container_isolated-4]        Waiting on external lifecycle transitions to activate
[component_container_isolated-4]        See https://design.ros2.org/articles/node_lifecycle.html for more information.
[component_container_isolated-4] [INFO] [1788180208.497178577] [amcl]: Subscribed to map topic.
[INFO] [launch_ros.actions.load_composable_nodes]: Loaded node '/behavior_server' in container '/nav2_container'
[component_container_isolated-4] [INFO] [1788180208.507553087] [nav2_container]: Load Library: /opt/ros/humble/lib/libbt_navigator_c
ore.so
[component_container_isolated-4] [INFO] [1788180208.518106992] [nav2_container]: Found class: rclcpp_components::NodeFactoryTemplate
<nav2_bt_navigator::BtNavigator>
[component_container_isolated-4] [INFO] [1788180208.518248499] [nav2_container]: Instantiate class: rclcpp_components::NodeFactoryTe
mplate<nav2_bt_navigator::BtNavigator>
[component_container_isolated-4] [INFO] [1788180208.532837972] [lifecycle_manager_localization]: Activating map_server
[component_container_isolated-4] [INFO] [1788180208.533643940] [map_server]: Activating
[component_container_isolated-4] [INFO] [1788180208.539530009] [map_server]: Creating bond (map_server) to lifecycle manager.
[component_container_isolated-4] [INFO] [1788180208.544780418] [amcl]: Received a 462 X 372 map @ 0.050 m/pix
[component_container_isolated-4] [INFO] [1788180208.557783428] [bt_navigator]:
[component_container_isolated-4]        bt_navigator lifecycle node launched.
[component_container_isolated-4]        Waiting on external lifecycle transitions to activate
[component_container_isolated-4]        See https://design.ros2.org/articles/node_lifecycle.html for more information.
[component_container_isolated-4] [INFO] [1788180208.558110058] [bt_navigator]: Creating
[INFO] [launch_ros.actions.load_composable_nodes]: Loaded node '/bt_navigator' in container '/nav2_container'
[component_container_isolated-4] [INFO] [1788180208.570903432] [nav2_container]: Load Library: /opt/ros/humble/lib/libwaypoint_follo
wer_core.so
[component_container_isolated-4] [INFO] [1788180208.576971360] [nav2_container]: Found class: rclcpp_components::NodeFactoryTemplate
<nav2_waypoint_follower::WaypointFollower>
[component_container_isolated-4] [INFO] [1788180208.577095139] [nav2_container]: Instantiate class: rclcpp_components::NodeFactoryTe
mplate<nav2_waypoint_follower::WaypointFollower>
[component_container_isolated-4] [INFO] [1788180208.596090780] [waypoint_follower]:
[component_container_isolated-4]        waypoint_follower lifecycle node launched.
[component_container_isolated-4]        Waiting on external lifecycle transitions to activate
[component_container_isolated-4]        See https://design.ros2.org/articles/node_lifecycle.html for more information.
[component_container_isolated-4] [INFO] [1788180208.600804537] [waypoint_follower]: Creating
[INFO] [launch_ros.actions.load_composable_nodes]: Loaded node '/waypoint_follower' in container '/nav2_container'
[component_container_isolated-4] [INFO] [1788180208.615330426] [nav2_container]: Load Library: /opt/ros/humble/lib/libvelocity_smoot
her_core.so
[component_container_isolated-4] [INFO] [1788180208.619849011] [nav2_container]: Found class: rclcpp_components::NodeFactoryTemplate
<nav2_velocity_smoother::VelocitySmoother>
[component_container_isolated-4] [INFO] [1788180208.619962806] [nav2_container]: Instantiate class: rclcpp_components::NodeFactoryTe
mplate<nav2_velocity_smoother::VelocitySmoother>
```

4.4 APP Navigation Operation

1. **Connect to Robot**:
   Open ROSRobot APP, select navigation mode, enter robot IP address and port `9090`, VideoTopic select `/usb_cam/image_raw/compressed`, and click **【Connect】**.

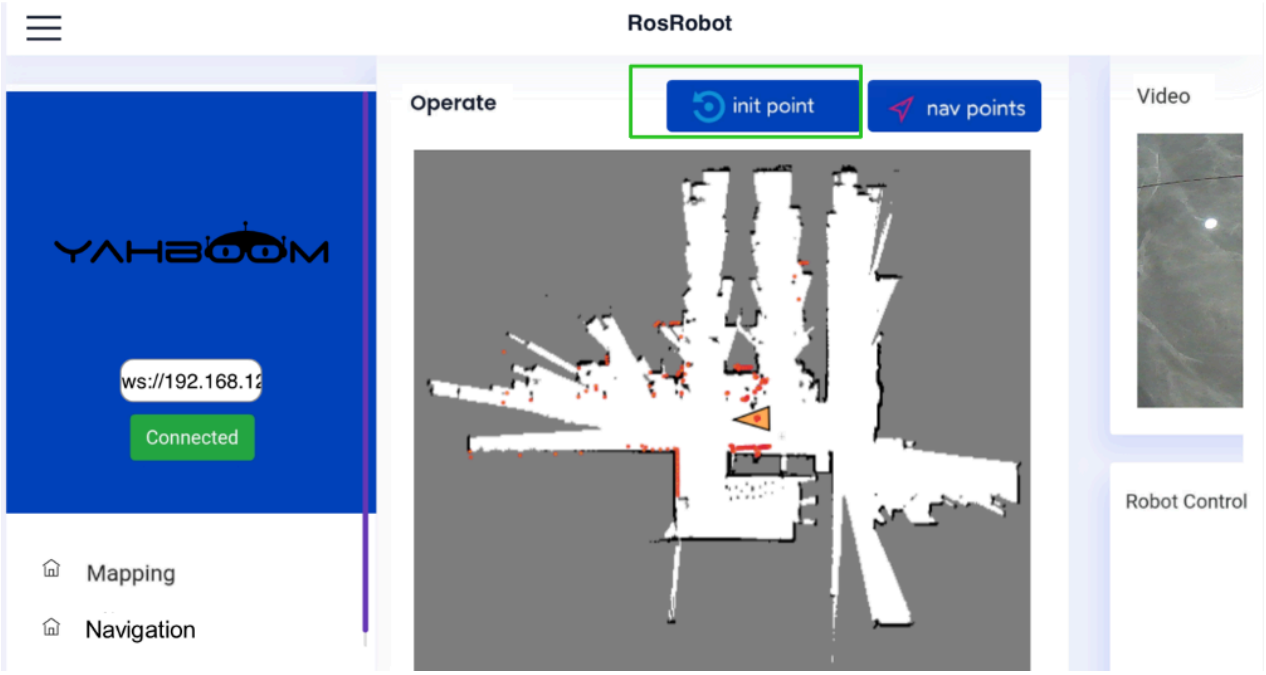


2. **Enter Navigation Main Interface**:
   After successful connection, the APP automatically loads the map and robot pose:

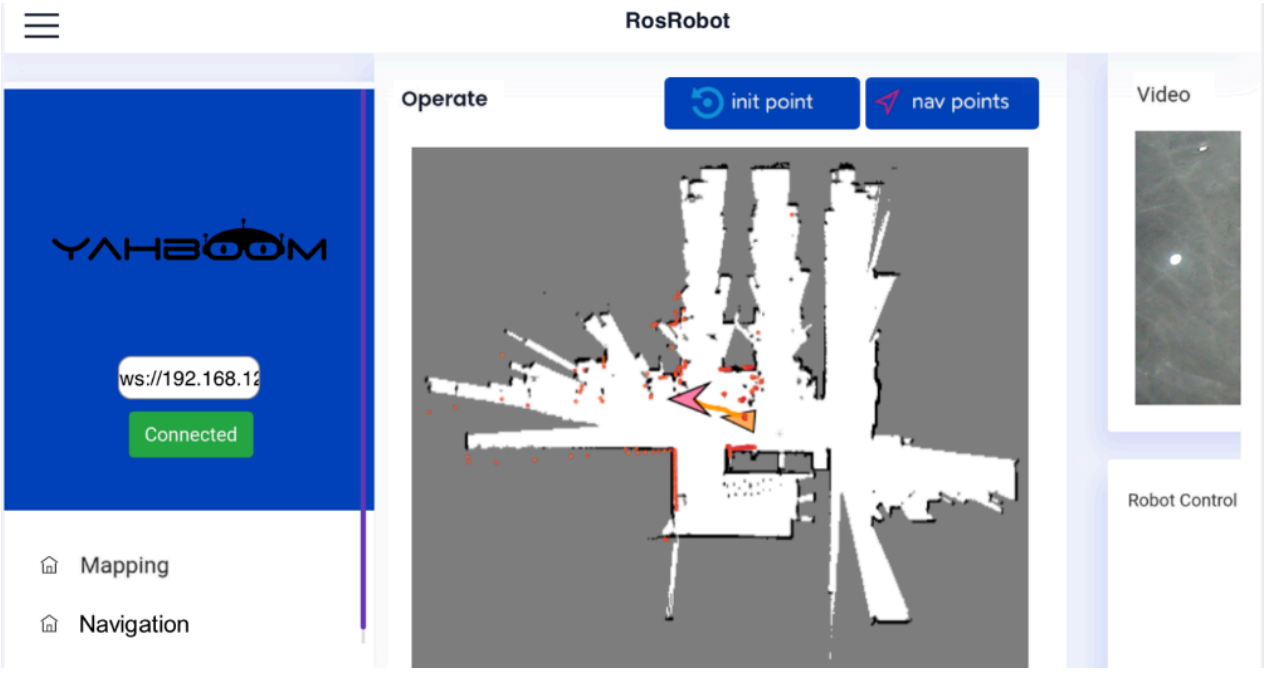


3. **Set Initial Point (2D Pose Estimate)**:

Click **【Set Initial Point】**, click at the actual physical position of the robot on the phone's map screen and pull out the chassis heading arrow. Observe that the red/green scanning point cloud on the map coincides with the black obstacle contours, representing successful initial pose alignment.



4. **Set Navigation Goal Point (2D Goal Pose)**:
   Click **【Set Navigation Point】**, click in the traversable free area on the map and set the goal heading. The robot immediately plans an optimal global path, autonomously controls steering and forward movement, smoothly avoids obstacles, and arrives at the destination.



> **Note**
>
> When you need to end the program, press `Ctrl+C` in each terminal window to stop the program.

5. Core Source Code and Communication Configuration Analysis

5.1 APP Communication Bridge Architecture Analysis

The mobile APP achieves full-duplex communication with the ROS 2 system through standard WebSocket protocol:

1. `rosbridge_websocket`:
    - Listens on port `9090`, responsible for transparently forwarding ROS 2 topics, services, and mobile APP JSON messages in both directions.
    - APP-published control commands are transformed into `/cmd_vel` to drive the chassis; APP-set pose points are transformed into `/initialpose` and `/goal_pose` and passed to Nav2.
2. `robot_pose_publisher`:
    - Real-time listens to the `map -> base_footprint` in the TF transformation tree, calculates the robot's absolute coordinates and heading in the global map, and high-frequency sends to the APP interface via `/robot_pose` topic to draw the robot icon.
3. `laserscan_to_point_publisher`:
    - Converts lidar scan data into a compact point cloud format, providing real-time obstacle point cloud outline for APP rendering.

APP Navigation Launch Architecture Configuration (`navigation_teb_app_launch.xml`):

```
<launch>
    <!-- Start rosbridge websocket bridge service -->
    <include file="$(find-pkg-share
rosbridge_server)/launch/rosbridge_websocket_launch.xml"/>

    <!-- Start lidar point cloud publishing node -->
    <node name="laserscan_to_point_publisher" pkg="laserscan_to_point_publisher"
exec="laserscan_to_point_publish"/>

    <!-- Start TEB navigation core -->
    <include file="$(find-pkg-share yahboomcar_nav)/launch/nav2/navigation_teb_launch.py"/>

    <!-- Start robot map pose publishing node -->
    <include file="$(find-pkg-share
robot_pose_publisher_ros2)/launch/robot_pose_publisher_launch.py"/>
</launch>
```

5.2 APP Map Save Service Source Analysis

Source Code Path:

```
$HOME/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_app_save_map/yahboomcar_app_save_map/y
ahboom_app_save_map.py
```

```
class MinimalService(Node):

    def __init__(self):
        super().__init__('minimal_service')
        # Create ROS 2 service to receive APP map save requests
        self.srv = self.create_service(WebSaveMap, 'yahboomAppSaveMap',
self.add_three_ints_callback)

    def run_shellcommand(self, *args):
        '''Execute shell command and run map_saver_cli in background'''
        args = sum([(arg if type(arg) == list else [arg]) for arg in args], [])
        possess = subprocess.Popen(args, stdout=subprocess.PIPE)
        return possess

    def add_three_ints_callback(self, request, response):
        map_name = request.mapname
        workspace_src = Path.home() / 'yahboomcar_ros2_ws' / 'yahboomcar_ws' / 'src'
        map_path = str(workspace_src / 'yahboomcar_nav' / 'maps' / map_name)
        database_path = workspace_src / 'yahboomcar_app_save_map' /
'yahboomcar_app_save_map' / 'xgo.db'

        # Record map name and timestamp information into local SQLite database
        now = datetime.datetime.now()
        str_time = now.strftime("%Y-%m-%d %H:%M:%S.%f")
        map_id = hashlib.md5((str_time + map_name).encode()).hexdigest()
        response.response = request.mapname

        try:
            conn = sqlite3.connect(str(database_path))
            c = conn.cursor()
            c.execute("INSERT INTO xgo_map (map_name, map_id, map_path) VALUES (?, ?, ?)",
(map_name, map_id, map_path))

            # Call Nav2 official map_saver_cli node to save .yaml and .pgm raster map
            self.run_shellcommand(
                'ros2', 'run', 'nav2_map_server', 'map_saver_cli',
                '-f', map_path,
                '--ros-args',
                '-p', 'save_map_timeout:=10.0',
                '-p', 'map_subscribe_transient_local:=false',
            )
        except sqlite3.IntegrityError as e:
            response.response = {"msg": str(e)}
            return response

        self.get_logger().info('Map saved successfully: %s -> %s' % (map_name, map_path))
        return response
```

- This node creates the `yahboomAppSaveMap` service (type is `WebSaveMap`). When the APP clicks Save Map, it sends a request with the map name; while writing map metadata to SQLite database, the node asynchronously calls `nav2_map_server`'s `map_saver_cli` command-line tool to persistently save the current global occupancy raster map as `maps/<map_name>.yaml` and `maps/<map_name>.pgm`.

5.3 Core Communication Parameters and Port Description

Communication Module	Service/Topic/Port	Default Config Value	Core Function Description
WebSocket	Listening port	`9090`	ROSBridge service port, handles APP JSON control commands and topic broadcast
Video Streaming	Compressed image topic	`/usb_cam/image_raw/compressed`	Low-latency compressed video stream provided to APP for rendering
Pose Publishing	Topic name	`/robot_pose`	Publishes robot's real-time position and attitude in global map coordinate frame
Point Cloud Publishing	Topic name	`/scan_point`	Lidar point cloud coordinate transformation publishing, used by APP to draw obstacle point cloud
Map Save	Service name	`yahboomAppSaveMap`	Receives map name sent by APP and calls `map_saver_cli` in background to save